Language Models as Reward Functions for Reinforcement Learning

Allen Dufort, Zhinuo Wang, Subham Kumar Das, Yu Nie, and Dhruv Gada

Computer Science Department, Brown University

Abstract :

In this article, we propose a framework in which large language models (LLMs) generate reward signals by evaluating agent transitions against human-readable task descriptions. We instantiate this approach in two benchmark environments—Frozen Lake and Blackjack—within a Q-learning paradigm, comparing LLM-derived rewards in no-memory, long-term, short-term, and summary-memory modes against traditional numeric rewards trained over 500 episodes for Blackjack and 400 episodes for Frozen Lake. Our results highlight the potential of language-driven rewards to provide a more flexible and intuitive mechanism for training RL agents toward generalizable behaviors, advancing the path toward AGI.

Index Terms: Artificial General Intelligence, Artificial Intelligence, Large Language Model, Natural Language, Reinforcement Learning

1 Introduction

Reinforcement learning (RL) has played a key role in advancing agents to excellence in specific tasks. However, training RL agents typically depends on particular mathematical reward models tailored to particular tasks. This makes achieving Artificial General Intelligence (AGI) difficult, as these models need significant modification or rebuilding in different contexts.

In this paper, we dive into the questions "How can reward functions be generated/approximated by a large language model (LLM) based on human preferences prompts?" and "Can these language-based reward functions perform better than a mathematical reward function?". We propose using LLMs to generate reward functions in order to provide a more generalized and flexible framework for training RL agents across different tasks. The project will prompt an LLM with an RL task to reward the RL agent based on its understanding of the task. This allows for reward shaping without explicitly specifying the mathematical equation for the reward function.

The source code can be found here: https://github.com/lost-particles/echoLLM.

1.1 Contribution

This paper aims to contribute the following:

- Provide a generalized way to provide reward functions to train RL agents
- Integrate LLMs into the model training process, advancing the development toward AGI
- Make AI more intuitive and accessible for people with a lower technological background

2 Background

2.1 Artificial General Intelligence

Artificial General Intelligence (AGI) refers to systems that can understand, learn, and apply knowledge in a wide variety of domains at a level comparable to (or surpassing) that of humans. Unlike narrow AI, which excels at a single task (e.g., image classification or game play), AGI strives for broad adaptability and the capacity to transfer learning across tasks.

LLMs trained in massive text corpora have demonstrated surprising generality across language understanding, code generation, and reasoning benchmarks. This "foundation model" approach suggests that sufficiently large and versatile architectures could serve as one of the core building blocks for future AGI systems.

2.2 Reinforcement Learning and Reward Shaping

Reinforcement Learning (RL) is typically formalized as a Markov Decision Process (MDP)

$$M = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle,$$

where an agent in state $s \in \mathcal{S}$ selects action $a \in \mathcal{A}$, transitions to s' according to $P(s' \mid s, a)$, and receives reward $R(s, a)$. A core challenge in RL is designing the reward function so that it both encodes the true task objective and provides sufficiently dense feedback to guide learning.

Reward shaping addresses this by augmenting the base reward with an auxiliary term:

$$R'(s, a, s') \;=\; R(s, a) \;+\; F(s, a, s')\,.$$

Here, F is chosen to inject domain knowledge or heuristics (for example, "give a small bonus for moving closer to the goal"). The process of augmenting or modifying the reward signal to guide the agent toward desired behaviors does not require changing the underlying optimal policy. Classical approaches include potential-based shaping functions (Wiewiora, 2003) and demonstration-based priors (Brys et al., 2015). In our project, we replace the base reward and any manual shaping term with a single, language-based reward model: an LLM prompted to score each transition on a scale [0,1].

2.3 Related Works

Waytowich et al.'s paper provides "a technique that can provide the benefits of reward shaping using natural language commands" (Waytowich et al., 2019). This paper was instrumental in forming the theoretical basis of this project: the idea that you can use LLMs to provide a reward function to an RL agent.

Kwon et al. explore how to simplify reward design by prompting a large language model (e.g., GPT-3) as a proxy reward function. Users provide a natural-language prompt before training; at each step, the LLM evaluates the agent's behavior against this prompt and returns a scalar reward. They show that RL agents

trained with the LLM-derived reward closely align with user objectives and outperform agents using reward functions learned via supervised learning (Kwon et al., 2023).

In an article by Brian Buntz, he discusses how to use Q-learning to train RL agents to do a task. Additionally, his article provides a tutorial on how to do this with the Frozen Lake Challenge (Buntz, 2024). We used the code in this article to provide a baseline performance of Q-Learning and a foundation for our code. Our code differs from Buntz' by using an LLM as a reward function instead of a numerical algorithm.

3 Methods

To test whether language-based reward functions outperform mathematical ones, we designed 2 experiments where an LLM replaced a traditional reward function. In each experiment, we prompt the LLM with a task and goal and then use its output to guide the agent's behavior. In our setup, we used a Q-learning framework as described in Buntz, 2024, but replaced the standard reward function with responses from an LLM. Both experiments were carried out over 500 episodes to ensure that the results were statistically significant.

3.1 Frozen Lake

For the first experiment, we selected the OpenAI Gymnasium Frozen Lake game as our environment. We chose this environment because it was relatively small, with a 4x4 grid (as shown in Figure 1) and 4 actions (up, down, left, right), but still required skill to succeed.

The LLM was prompted with a description of the agent's current state, next state, and action, and asked to provide feedback in natural language alongside a rating from -5.0 (very bad) to 5.0 (excellent).

Figure 1. Diagram of RL agent interacting with Frozen Lake game

Several key considerations were carefully evaluated to ensure the LLM produced successful results; these are outlined below.

Spatial Pattern: One key consideration was including a textual pattern to spatially represent the grid. This was done so that the LLM could understand the spatial arrangement of these tiles with respect to each other and how the agent moves in the environment. The 4x4 grid is mapped as follows (state numbers):

```
 0  1  2  3
 4  5  6  7
 8  9 10 11
12 13 14 15
```

In addition to that, the LLM also has to understand which states are the hole in the lake (to be avoided), the goal (to aim for), and the start point. To accomplish this, we added a mapping between the states to the type of tiles each state represents. The layout of the grid is:

```
S F F F
F H F H
F F F H
H F F G
```

where, S = Start point, F = Frozen lake, H = Hole and G = Goal

LLM Prompt Design: For the LLM to learn, to properly score the moves taken by the RL agent, we used Few Shot Learning by adding examples of moves along with expected scores to show the difference between very bad, bad, neutral, good, and excellent moves. One example of that looks as such:

```
"### Examples:\n\n"
"1. Very bad move (fell into a hole):\n"
"- Recent Transitions:\n"
"  1. State 5 → [down] → State 9\n"
"- Agent: I am at state 5.\n"
"- Environment: You moved down to state 9 (a H tile).\n"
"- Note: State 9 is a hole. The episode ends here.\n"
"- Distance from current state to goal
  (state 15): 6 steps.\n"
"- Response: -5.0\n\n"
```

Initially, the LLM-trained RL agent would constantly get stuck in an infinite loop consisting of a few safe states (Frozen tiles) and never exit the episode. This was happening because of two main reasons :

- **The agent lacked sufficient incentive to explore alternative actions**, which led it to consistently follow the paths discovered during the initial training episodes. To encourage broader exploration, we initialized the Q-table with a value of 1.0 for all valid "Frozen" tiles instead of the default 0.0. This adjustment promotes exploration by making unexplored states appear more rewarding. Conversely, the Q-values for the "Hole," "Start," and "Goal" tiles were set to 0.0, reflecting that no further actions should be taken from these states.
- **In the absence of negative rewards, the agent was prone to entering and remaining in loops consisting of safe states**, as the accumulated reward monotonically increases with each step, effectively preventing episode termination. To mitigate this pathological behavior, we redefined the reward function to support both positive and negative values. We further leveraged an LLM to dynamically assign negative rewards upon detecting loop-like behavior in the agent's trajectory. To enable this, we maintain a sliding window over

the agent's recent state-action history, which is summarized and provided as input to the LLM. This temporal context allows the model to infer cyclic patterns and penalize the agent accordingly, thereby encouraging exploration and reducing the likelihood of infinite looping.

Manhattan Distance: Also known as the *L1 distance* or *taxicab distance*, it is defined as the sum of the absolute differences of the coordinates of two points.

For two points $A = (x_1, y_1)$ and $B = (x_2, y_2)$, the Manhattan distance D is given by:

$$D = |x_1 - x_2| + |y_1 - y_2|$$

In n-dimensional space, for vectors $\mathbf{a} = (a_1, a_2, \ldots, a_n)$ and $\mathbf{b} = (b_1, b_2, \ldots, b_n)$, the distance generalizes to:

$$D = \sum_{i=1}^{n} |a_i - b_i|$$

Example: For points $A = (1, 2)$ and $B = (4, 6)$,

$$D = |1 - 4| + |2 - 6| = 3 + 4 = 7$$

This distance metric is commonly used in grid-based pathfinding (e.g., A* on 2D grids) and in machine learning tasks where only axis-aligned movements are allowed (Horizontal and Vertical movements in our frozen lake case).
At each timestep, the LLM is provided with the Manhattan distance to the goal state, enabling it to estimate its progress toward completing the Frozen Lake task. This augmentation was necessary, as in the absence of explicit distance information, the model exhibited reduced effectiveness in discerning the relative value of available actions (i.e., left, right, up, down) with respect to long-term reward maximization.

Distinction on Episode Endings: Episodes can terminate in one of three ways: (1) the agent reaches the designated goal state, (2) the agent falls into a hole, or (3) the episode is forcefully terminated after exceeding 1000 steps, which serves as a safeguard against infinite loops. It is essential for the LLM to distinguish among these termination conditions, as they vary in desirability. Specifically, reaching the goal is considered a successful outcome and should be positively reinforced, whereas termination due to falling into a hole or exceeding the step limit should be penalized. Furthermore, among successful episodes, those completed in fewer steps should be rewarded more heavily, incentivizing both correctness and efficiency in policy learning.

Use of Memory: We utilized conversational history with the LLM, controlled via the context length parameter (set to 4096 in our case), which is included with each API call. This enables the model to reference prior interactions, allowing it to maintain continuity and avoid repeating previous mistakes.

3.2 Blackjack

For the second experiment, we selected the Blackjack environment from OpenAI Gymnasium. Blackjack is a card game where the agent must decide whether to `hit` (draw another card) or `stick` (end its turn), aiming to get a hand value as close to 21 as possible without going over. The state is defined by a tuple `(player sum, dealer card, usable ace)`, and the agent must learn when to take or avoid risk depending on the state.

We compared two types of reward functions:

- **Real Reward:** The default numeric reward from the environment (+1 for a win, 0 for a draw, -1 for a loss).
- **LLM-based Reward:** A language model (LLM) assigns a floating-point score in the range [0.0, 1.0] based on its evaluation of the agent's move. This replaces the native reward for Q-learning.

LLM Prompt Design The LLM is prompted with a detailed instruction and several few-shot examples to learn how to rate moves. Each example is a single transition in the format:

```
(player sum, dealer card, usable ace) → action →
                    next_state
```

along with a short explanation and a numeric score:

```
(13, 10, False) → hit → (23, 10, False) # Busted → 0.0
(20, 10, False) → stick → (20, 10, False) # Safe stand → 1.0
(12, 2, False) → stick → (12, 2, False) # Too passive → 0.3
(16, 10, False) → hit → (18, 10, False) # Risk paid off → 0.8
```

The instruction explicitly told the model to:

- Rate actions on a scale from 0.0 (bad) to 1.0 (excellent).
- Consider whether the move resulted in busting, risk-taking, or efficient play.
- Output only a single decimal number for evaluation consistency.

LLM Memory Modes To evaluate how context affects LLM reward generation, we defined three types of memory usage during scoring:

- **Adaptive Memory:** Our system allows dynamic adjustment of the context length to simulate either short-term or long-term memory. In the Blackjack environment, we use a short-term memory setting (**short-term memory**), as the decision-making process is relatively simple and does not require extended temporal context. In contrast, the Frozen Lake environment employs **long-term memory** due to the increased complexity involved in tracking prior actions, recognizing loops, and avoiding repeated failures such as falling into holes. In the Frozen Lake environment, we maintain a history of previous interactions with the LLM, forming a persistent conversational context. This history is dynamically trimmed based on the specified context length, while preserving the initial static prompt containing the game rules. Notably, this static prompt is excluded from the context length budget, ensuring consistent access to the task specification across all interactions. In the case of Blackjack, the LLM is provided with a window of recent state-action transitions, enabling it to identify local behavioral patterns such as loops, repeated mistakes, or high-risk strategies. These transitions are presented as:

  ```
  Recent state-action transition for Blackjack:
  (14, 10, False) → hit → (18, 10, False)
  ```

```
(18, 10, False) → stick → (18, 10, False)
```

- **Summary Memory ("summary"):** For the Frozen Lake environment, the actions taken by the LLM over the last 20 episodes are summarized and incorporated into the prompt as such:

  "### Recent Transitions:
  1. State 0 → [right] → State 1
  2. State 1 → [right] → State 2
  3. State 2 → [down] → State 6
  4. State 6 → [left] → State 5"

  For the Blackjack environment, the agent's full trajectory is summarized every 100 episodes into a natural language paragraph by the LLM, which is then appended to each prompt in the following episodes. For example:

  "Summary: The agent played conservatively, standing early on strong hands but taking calculated risks on weak hands."

- **No Memory ("none"):** The LLM receives only the current move transition with no historical context. This tests whether scoring is possible based solely on a single state-action pair.

Training Setup: All variants (real and LLM-based reward) were trained using Q-learning for 400 episodes each, using the following update rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

where r was either the LLM score or the environment's actual reward. We used an ϵ-greedy strategy with decay to encourage early exploration and later exploitation.

3.3 Evaluation

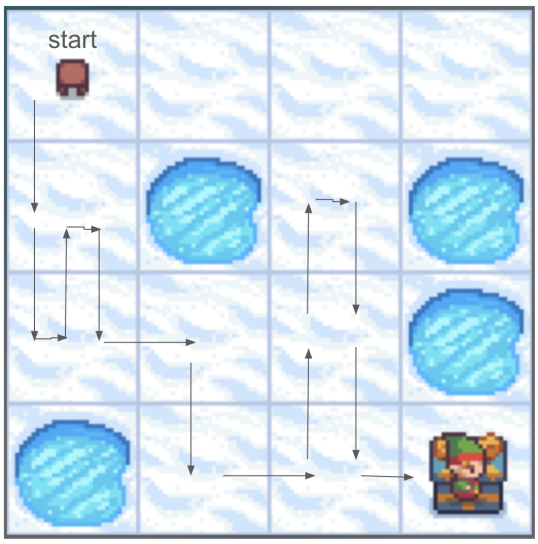
To assess the performance of each experiment, we analyzed the graphs representing the reward per episode. We then ran 500 episodes(for Blackjack) and 400 episodes(for Frozen Lake) of gameplay with 2 agents, one trained using traditional Q-learning methods and one trained with our language-based reward function. We measured success by the time it took each episode to run and the number of times each type of agent won in the episodes.

4 Results

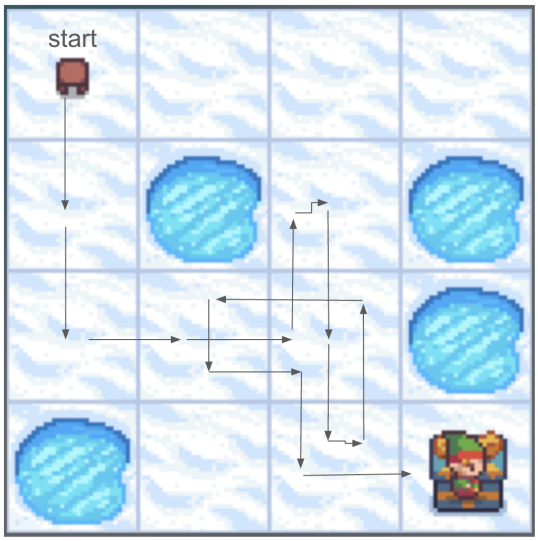
4.1 Frozen Lake

Our model was trained for 400 episodes and, when visualized, the process within 500 episodes is capable of completing around 20 to 25 episodes. Examples of our train agent's traces are presented in Figure 2a, Figure 2b, and Figure 2c. GIFs demonstrating successful runs are available in our GitHub repository (in the results folder), as referenced in the Introduction section above.

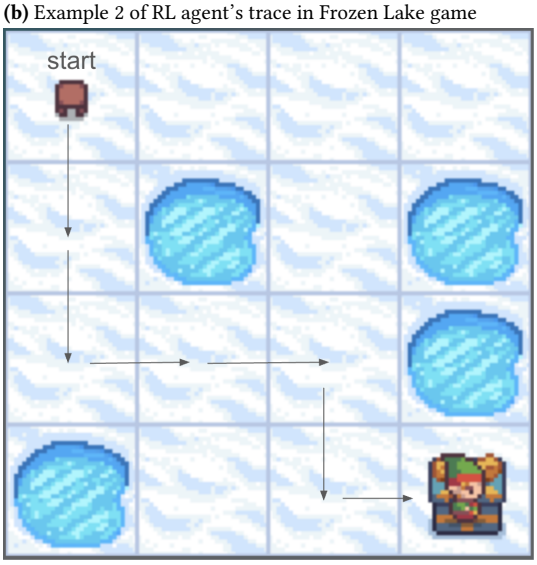
Figure 3 presents a comparison of success rates between the LLM-based and numerically-defined reward functions in the Q-Learning framework. While the LLM-based reward yields a lower success rate relative to the conventional mathematical reward function, it nonetheless demonstrates the feasibility of training an



(a) Example 1 of RL agent's trace in Frozen Lake game



(b) Example 2 of RL agent's trace in Frozen Lake game



(c) Example 3 of RL agent's trace in Frozen Lake game

agent to successfully complete the FrozenLake environment. Potential avenues for improving the LLM-guided reward formulation are addressed in the subsequent *Discussion* section.

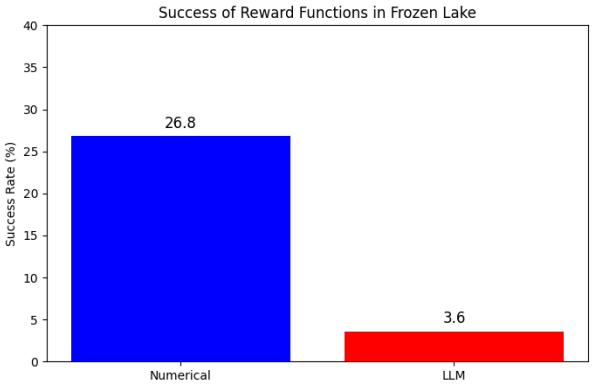


Figure 3. Frozen Lake: Success of Reward Functions

4.1.1 Key Observations Figure 4 represents the aggregated rewards given by the LLM over all the moves performed in each episode. The following can be observed :

- **Consistent Gradient in Reward Signal:** The LLM provides a graded and consistent reward signal across episodes – with clear variation rather than binary success/failure. This is a positive sign for RL training, as it provides the agent with informative feedback to learn from.
- **Pattern Recognition is Working:** The distribution shows distinguishable clusters of reward totals, suggesting that the LLM responds differently to qualitatively different behaviors (e.g., looping, falling in holes, or risk-taking). That's encouraging – the LLM appears sensitive to episode-level structure.
- **Few Catastrophic Failures:** Only a small number of episodes fall in the extreme negative range (< -60), which may mean the agent is avoiding the worst-case behaviors most of the time, even if suboptimal.
- **Potential for Improvement is Clear:** Since most rewards fall in a manageable range (say, -40 to -10), this creates a clear margin for upward improvement during training, especially if guided exploration or curriculum learning is added.

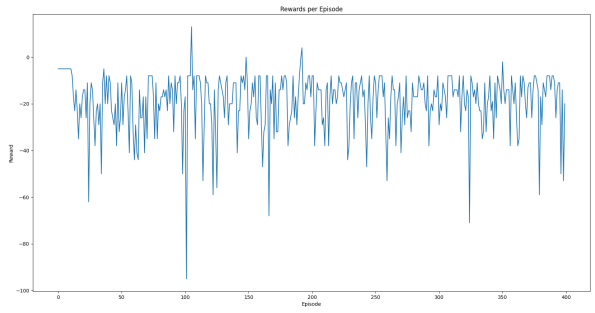


Figure 4. Frozen Lake: Distribution of LLM Rewards across episodes

4.2 BlackJack

Figure 5 summarizes the total number of wins, draws, and losses for each method after 500 episodes of training. We observe that while the performance of each strategy varies slightly, LLM-based reward shaping shows competitive and in some cases superior results compared to traditional real-reward training.

In this case, the **LLM-Summary** variant achieved the highest number of wins (181), followed closely by **LLM-Short** (174) and **LLM-None** (167), all outperforming the real reward baseline (153 wins). Although the LLM-based approaches did not always reduce the number of losses, they consistently demonstrated stronger win rates overall. Interestingly, the **LLM-Short** configuration had the fewest number of draws (12), suggesting that short-term memory encourages more decisive play, either pushing the agent toward a win or resulting in a loss. Conversely, the real-reward baseline led to a comparatively higher draw count (39), indicating more conservative play.

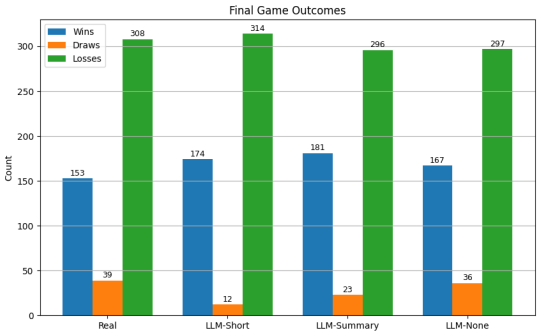


Figure 5. Final win/draw/loss counts across Real and LLM-based reward methods. Values are labeled on each bar.

4.2.1 Key Observations

- LLM-based rewards can provide effective guidance for policy learning in Blackjack, especially when augmented with short-term or summary memory.
- The LLM-Summary model achieved the best win performance, potentially due to its exposure to holistic strategic summaries.
- While individual outcome counts may vary slightly between runs due to the stochastic nature of Blackjack and training initialization, the LLM-based methods consistently match or outperform the real-reward baseline in our experiments.

These results suggest that LLMs, when used as reward signal generators, can be competitive with or even superior to traditional numeric rewards in guiding policy learning. While outcomes may vary slightly across different runs, the consistent trend observed here supports the utility of LLM-based reward shaping in reinforcement learning tasks.

Post-Training Policy Evaluation To assess the performance of the final learned policies, we conducted 100 test episodes for each trained agent using greedy action selection (i.e., $\epsilon = 0$). The results are shown in Table 1. Among the evaluated strategies, the **LLM-Summary** policy achieved the highest win rate of 36%, slightly outperforming the Real Reward policy (33%). The LLM-

None variant performed the worst with only 26% wins, suggesting that memory context (summary or short) plays a crucial role in guiding the LLM during reward shaping.

Table 1
Test performance over 100 episodes per method (greedy policy)

Method	Wins	Draws	Losses	Win Rate
Real Reward	33	17	50	33.00%
LLM-Summary	36	3	61	36.00%
LLM-Short	36	5	59	36.00%
LLM-None	26	6	68	26.00%

5 Discussion

5.1 Prompt Shaping

During the study, we discovered that prompt-feeding to the LLM plays a crucial role in guiding its behavior and requires careful modifications. Initially, the LLM failed to adhere to its response format, even when we formatted the prompt as "How good was this move on a scale from 0 (very bad) to 1 (excellent)? Respond with a single number only." While the instructions were explicit and easily understandable to fluent English users, some LLMs nonetheless generated responses including unsolicited explanations or additional formatting not requested in the prompt. In order to solve this problem, we expanded our prompt, providing a lot of examples to feed into the LLM so that it knew what the response should be like in any given scenario.

We also found that for complex game rules, providing examples helps the LLM understand and perform. In the Blackjack environment, we categorized the actions and corresponding results and rewards, and fed them to the prompt. Instead of simply giving the rules, we gave examples as below:

```
"(13, 10, False) → hit → (23, 10, False)
# Busted → 0.0"
"(20, 10, False) → stick → (20, 10, False)
# Safe stand → 1.0"
"(12, 2, False) → stick → (12, 2, False)
# Too passive → 0.3"
"(16, 10, False) → hit → (18, 10, False)
# Risk paid off → 0.8"
"(18, 10, False) → hit → (24, 10, False)
# Unnecessary risk → 0.1"
"(19, 9, False) → stick → (19, 9, False)
# Correct stand → 0.9"
```

Similarly, Few examples given to the LLM to showcase the reward mechanism for Frozen Lake are shown below :

```
"2. Bad move (returned to a previously visited state
 unnecessarily):\n"
    "- Recent Transitions:\n"
    "  1. State 1 → [left] → State 0\n"
    "  2. State 0 → [right] → State 1\n"
    "- Agent: I am at state 1.\n"
  "- Environment: You moved left to state 0 (a S tile).
    \n"
   "- Distance from current state to goal (state 15):
    6 steps.\n"
    "- Response: -3.0\n\n"

"8. Helpful move (progressed toward the goal while avoiding
 danger):\n"
    "- Recent Transitions:\n"
    "  1. State 1 → [right] → State 2\n"
    "  2. State 2 → [down] → State 6\n"
    "- Agent: I am at state 6.\n"
  "- Environment: You moved down to state 10 (a F tile).\n"
   "- Distance from current state to goal (state 15):
    2 steps.\n"
    "- Response: 3.5\n\n"
```

5.2 Rule-complexity vs. Action-space Size

During the study, we explored the use of LLMs to generate reward signals for two distinct environments: Frozen Lake and Blackjack. As mentioned previously, the agent has 4 different actions in Frozen Lake, whereas the agent in Blackjack only has two actions. Normally, a larger action space increases the difficulty of the game as it brings more potential solutions. However, we observed that lightweight LLM-based models, such as TinyLlama, performed consistently well in Frozen Lake but failed to provide meaningful reward differentiation in Blackjack. Those models often returned the same reward throughout the whole journey. After further studying this phenomenon, we discovered that certain LLM models do not fully understand the blackjack rules.

During this process, we learned that when using LLM to prompt the reward functions, prompts need to be well aligned with the LLM's training and comprehension so that it can fully understand the situational settings.

5.3 Overfitting

Another challenge we encountered is the tendency of LLMs to overfit. For example, in the Frozen Lake game, when there were no holes in the upper rows, the LLM consistently over-rewarded the agent so that it never went down to approach the goal. To fix this issue, we had to manually tell the LLM in the prompts to give penalties for cycling or backtracking. This shows that the prompts provided to LLMs should still be designed carefully according to the game settings.

5.4 Computational Limitations

Our most severe limitation was our computing power. For this project, we only had access to Brown University virtual machines and our own laptops, both with GPU and RAM limits. With many of the top LLMs having over 100 billion parameters, it was impossible to download and use the most advanced language models. We then had to use smaller, less advanced LLMs, significantly affecting our experiments.

6 Conclusion

6.1 Future Changes

With more time and resources available, we would like to utilize some of the more advanced LLMs like OpenAI's 4o model and have our agent trained in more sophisticated environments and game rules.

We would also get a computer system with more RAM, CPU,

and GPU space. This would allow us to download larger LLMs such as the 4o model. Having bigger models may help with the LLM's comprehension of the task, prompt, and the LLM's reasoning skills. This, in turn, could allow for a higher success rate.

6.2 Final Thoughts

By demonstrating that LLM-derived rewards can rival and even surpass traditional numeric functions in various tasks, our work underscores the viability of integrating natural language understanding into the core RL mechanisms. Although challenges in prompt design, model size, and computational resources remain, the proposed approach lays the groundwork for more human-aligned reward shaping without extensive manual engineering, especially when designing reward functions is impossible/impractical. Moving forward, exploring larger and more capable LLMs, refining memory integration strategies, and extending evaluations to complex, high-dimensional environments will be crucial steps toward fully realizing the promise of language-driven rewards in the pursuit of Artificial General Intelligence.

This project also has the capacity to make AI more accessible to people with low technological backgrounds. The natural language reward functions can allow people to train and specialize RL agents without even having to know about reward functions or other complex mathematical concepts. In general, the generalization of RL will allow people to develop AI and AGI just by *saying* what they want done.

Acknowledgements

This research received support from the Brown University course CSCI 2951X: Reintegrating AI, instructed by Professor George Konidaris.

References

Brys, Tim et al. (2015). "Reinforcement Learning from Demonstration through Shaping." In: *IJCAI*, pp. 3352–3358.

Buntz, Brian (Apr. 2024). *Slip and Slide into Reinforcement Learning with the Frozen Lake Challenge*. URL: https://www.rdworldonline.com/slip-and-slide-into-reinforcement-learning-with-the-frozen-lake-challenge/.

Kwon, Minae et al. (2023). "Reward design with language models". In: *arXiv preprint arXiv:2303.00001*.

Waytowich, Nicholas et al. (2019). *A Narration-based Reward Shaping Approach using Grounded Natural Language Commands*. arXiv: 1911.00497 [cs.AI]. URL: https://arxiv.org/abs/1911.00497.

Wiewiora, E. (Sept. 2003). "Potential-Based Shaping and Q-Value Initialization are Equivalent". In: *Journal of Artificial Intelligence Research* 19, pp. 205–208. ISSN: 1076-9757. DOI: 10.1613/jair.1190. URL: http://dx.doi.org/10.1613/jair.1190.